SE3004 Software Testing - Term Project

ShopEasy Reflection Report

Reflection on Testing Techniques and Bugs Revealed

Working through the testing techniques from Chapters 1 to 6 on the ShopEasy codebase was an eye-opening experience. During the Specification-Based (Task 1) and Structural Testing (Task 2), I mostly confirmed that the happy paths and basic boundaries worked as expected. The assert statements added in Task 3 (Design by Contract) were great for immediately stopping bad data, like negative prices, but they acted more like safety nets.

However, things got really interesting in Task 4 with Property-Based Testing. Jqwik completely surprised me by catching two specific bugs that my manual boundary checks missed. First, during the "Boundedness" property, it caught a floating-point precision error. The test generated a random base price of 0.01 and a tax rate of 0.44. Because of how Java handles double arithmetic, the actual result slightly deviated from the exact mathematical expectation (e.g., yielding something like 0.010044 instead of exactly 0.01004399...), causing my strict assertion to fail. I had to add a small tolerance (within(0.001)) to fix the test, which was a practical reminder of why we shouldn't use double for financial logic. Second, the "Cart Commutativity" property broke because jqwik generated two different products that happened to share the same randomly generated name/ID but had different prices. The ShoppingCart simply saw the matching ID and wrongly updated the quantity of the first item. I resolved this by modifying my custom @Provide method to assign a UUID.randomUUID().toString() to every generated product.

Most Effective Technique Per Unit of Effort

If I have to choose the most effective technique for the time and effort spent, it is definitely Property-Based Testing (Chapter 5). Writing the partition and boundary tests manually in Task 1 was tedious, and hunting down missing branches in the JaCoCo report required repetitive back-and-forth. On the other hand, for jqwik, I only had to write the high-level properties and a data provider. Once set up, the engine automatically fired thousands of randomized edge cases at my code. Finding that product ID collision and the floating-point bug took almost no extra effort on my part—the tool did the heavy lifting. The return on investment here was huge.

Mocking Trade-offs (Task 5)

Using Mockito to mock the InventoryService and PaymentGateway was essential for isolating the OrderProcessor. Mocking allowed me to test the pure orchestration logic very quickly without needing a live, populated database or a real credit card API. It enabled me to easily simulate catastrophic edge cases, like a rejected payment or partial inventory. However, the major trade-off is that it creates a blind spot by preventing true integration testing. My tests prove that OrderProcessor calls the charge() method, but they cannot verify if the system will actually crash over a slow network, or if the real database schema has changed. Mocking gives a false sense of security if not paired with broader tests.

Testing Pyramid Classification and Future Work

Looking at the Testing Pyramid from Chapter 1, my entire test suite sits firmly at the very bottom layer: Unit Tests. Every single test isolates a specific class, either by testing it on its own or by completely faking its dependencies. What is fundamentally missing from this test suite are Integration Tests and End-to-End (E2E) Tests.

If I had another week to work on this project, I would definitely step up the pyramid and focus on Integration Tests. I would want to replace the Mockito stubs with a real database environment (perhaps using Testcontainers) to see how the InventoryService actually handles reading and writing stock levels. Additionally, since this is a simulated e-commerce system, I would write concurrency tests. It would be highly critical to test what happens if two threads attempt to call OrderProcessor.process() simultaneously for the last remaining unit of a product to see if our system handles race conditions correctly.